Job 1 — La chaîne de compilation GCC

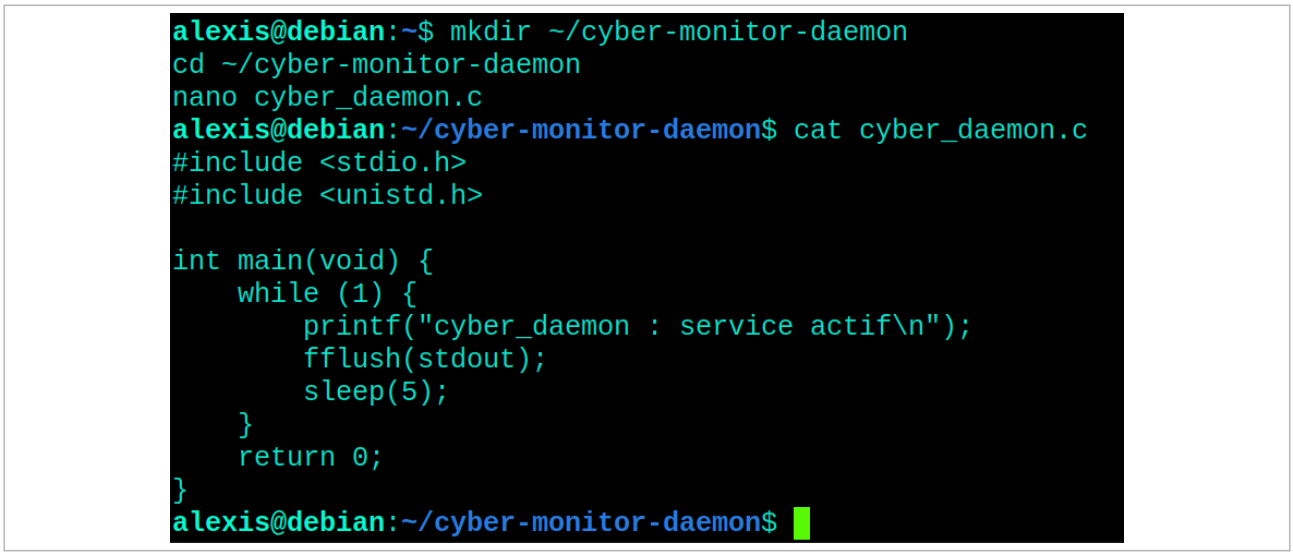
Compte rendu avec captures d'écran et réponses aux questions

Alexis Noiret · La Plateforme · Bachelor IT Cybersécurité · 27 octobre 2026

Environnement : Debian 13 (Trixie), GCC 14.2.0

Le programme : cyber_daemon.c

Programme C minimaliste qui tourne indéfiniment, affiche un message toutes les 5 secondes et force l'écriture immédiate de sa sortie.



Création du dossier et du fichier source, vérifié avec cat.

```
#include <stdio.h>
#include <unistd.h>

int main(void) {
    while (1) {
        printf("cyber_daemon : service actif\n");
        fflush(stdout);
        sleep(5);
    }
    return 0;
}
```

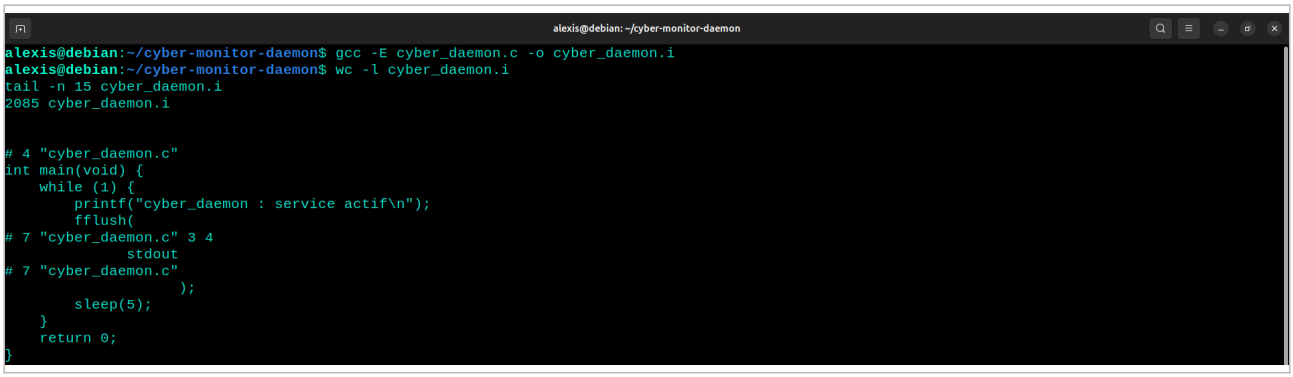
- **while (1)** : boucle infinie.
- **printf** : affiche le message ; **sleep(5)** : pause de 5 secondes.
- **fflush(stdout)** : vide le tampon de sortie. Sans cela, une sortie redirigée vers le journal (cas de systemd au Job 2) resterait en mémoire tampon et n'apparaîtrait pas dans les logs.

Vue d'ensemble de la chaîne

```
.c  --(gcc -E)-->  .i  --(gcc -S)-->  .s  --(gcc -c)-->  .o  --(édition de liens)-->  exécutable
```

1 — Le préprocesseur (flag -E)

```
gcc -E cyber_daemon.c -o cyber_daemon.i
```



Sortie de gcc -E : 2085 lignes dans le fichier .i, le code d'origine se trouve tout en bas.

Question : que s'est-il passé avec vos #include ?

Le préprocesseur a remplacé chaque ligne **#include** par le **contenu complet du fichier d'en-tête** correspondant (stdio.h, unistd.h et tous les en-têtes qu'ils incluent eux-mêmes). Le fichier est passé de 11 lignes à **2085 lignes**. Ces lignes ajoutées sont des déclarations (types, prototypes de fonctions comme printf, fflush, sleep). Le code écrit par l'utilisateur est inchangé et se trouve en fin de fichier.

Les lignes du type `# 4 "cyber_daemon.c"` sont des repères ajoutés par GCC (fichier et numéro de ligne d'origine, utiles pour les messages d'erreur). Ce n'est pas du code.

2 — La traduction en assembleur (flag -S)

```
gcc -S cyber_daemon.i -o cyber_daemon.s
```

```
alexis@debian:~/cyber-monitor-daemon$ gcc -S cyber_daemon.i -o cyber_daemon.s
cat cyber_daemon.s
        .file   "cyber_daemon.c"
        .text
        .section        .rodata
.LC0:
        .string "cyber_daemon : service actif"
        .text
        .globl  main
        .type   main, @function
main:
.LFB0:
        .cfi_startproc
        pushq   %rbp
        .cfi_def_cfa_offset 16
        .cfi_offset 6, -16
        movq    %rsp, %rbp
        .cfi_def_cfa_register 6
.L2:
        leaq    .LC0(%rip), %rax
        movq    %rax, %rdi
        call    puts@PLT
        movq    stdout(%rip), %rax
        movq    %rax, %rdi
        call    fflush@PLT
        movl    $5, %edi
        call    sleep@PLT
        jmp     .L2
        .cfi_endproc
.LFE0:
        .size   main, .-main
        .ident  "GCC: (Debian 14.2.0-19) 14.2.0"
        .section        .note.GNU-stack,"",@progbits
alexis@debian:~/cyber-monitor-daemon$
```

Contenu de cyber_daemon.s : assembleur x86-64.

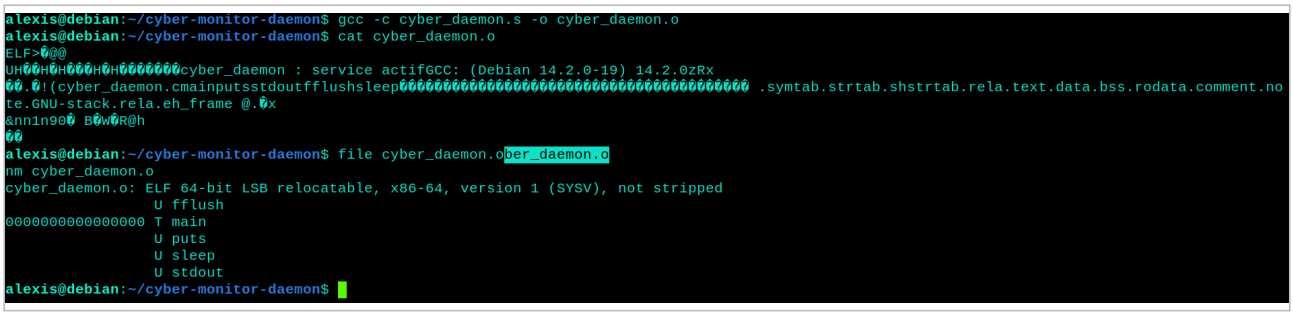
Question : reste-t-il la moindre trace du langage C ? À quoi ressemble ce que vous voyez ?

Il ne reste **aucune syntaxe C** : plus de while, de point-virgule, d'accolades ni de #include. Le fichier contient de l'**assembleur x86-64** : des instructions processeur lisibles par un humain (pushq, movq, leaq, call, jmp), une par ligne, plus des directives de l'assembleur (lignes commençant par un point).

- **.LC0 / .string** : le texte du message, rangé en zone de données en lecture seule (.rodata).
- **call puts@PLT** : GCC a remplacé printf par puts, plus rapide, car le texte est fixe et fini par un retour à la ligne.
- **call fflush@PLT** et **call sleep@PLT** (avec la valeur 5 placée dans %edi) : les deux autres appels du programme.
- **jmp .L2** : saut inconditionnel vers le début de la boucle. C'est l'équivalent assembleur de while (1).

3 — La génération du fichier objet (flag -c)

```
gcc -c cyber_daemon.s -o cyber_daemon.o
cat cyber_daemon.o
file cyber_daemon.o
nm cyber_daemon.o
```



cat sur le .o (binaire illisible), puis file et nm. Note : la commande file a été tapée avec une faute de frappe, sans conséquence sur le résultat.

Question : pourquoi ce fichier n'est-il toujours pas exécutable, alors qu'il est déjà en binaire ?

cat affiche des caractères illisibles : le fichier contient du code machine, pas du texte. Seules quelques chaînes sont lisibles (ELF, le message, la version de GCC, les noms de fonctions).

La commande **file** indique `ELF 64-bit LSB relocatable` : le fichier est **relogeable**, c'est-à-dire destiné à être assemblé avec d'autres morceaux, pas à être lancé seul.

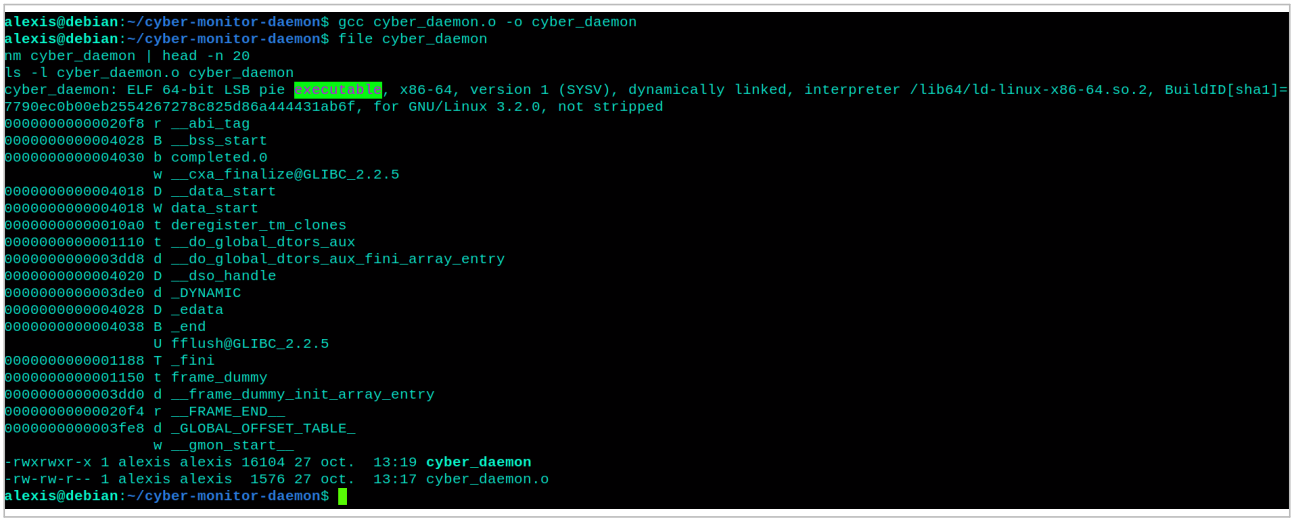
La commande **nm** donne la raison :

```
T main        -> defini dans ce fichier
U puts        -> non resolu (undefined)
U fflush      -> non resolu
U sleep       -> non resolu
U stdout      -> non resolu
```

Le .o appelle des fonctions (puts, fflush, sleep) dont le code n'est **pas** dans le fichier : elles se trouvent dans la bibliothèque C (libc). Il manque aussi le code de démarrage qui appelle main. Le fichier est donc du binaire, mais incomplet.

4 — L'édition de liens (liaison finale)

```
gcc cyber_daemon.o -o cyber_daemon
file cyber_daemon
ls -l cyber_daemon.o cyber_daemon
```



Après l'édition de liens : file indique executable, nm montre beaucoup plus de symboles, et la taille passe de 1576 à 16104 octets.

Question : qu'est-ce que l'édition de liens a ajouté par rapport au fichier objet précédent ?

Comparaison : le fichier objet pesait **1576 octets**, l'exécutable pèse **16104 octets**. `file` indique maintenant *pie executable, dynamically linked* au lieu de *relocatable*.

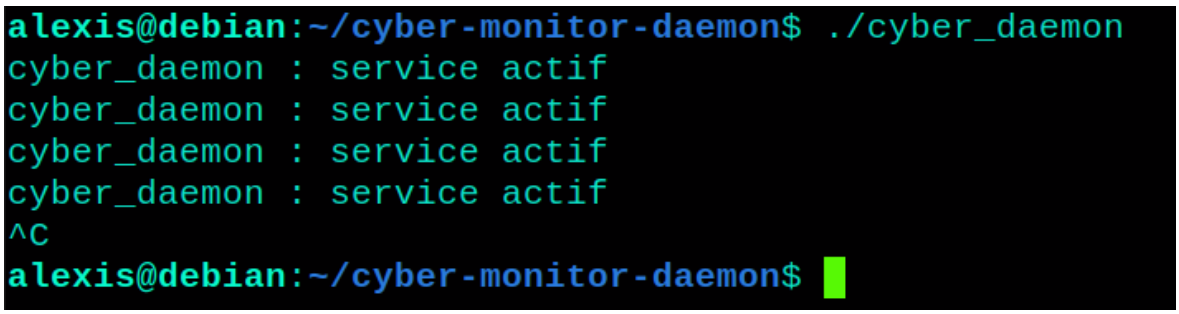
L'édition de liens a ajouté :

- le **code de démarrage** (_start, fourni par crt1.o) qui prépare le programme puis appelle main ;
- les **mécanismes de liaison avec la libc** (tables GOT et PLT, interpréteur dynamique /lib64/ld-linux-x86-64.so.2) pour que puts, fflush et sleep soient résolus au lancement ;
- l'**en-tête d'exécutable** et la mise en page finale (adresses, sections), qui permettent au noyau de lancer le fichier.

Le symbole `U fflush@GLIBC_2.2.5` reste marqué U : c'est normal, la fonction est fournie par la libc dynamiquement au moment de l'exécution.

Test final de l'exécutable

```
./cyber_daemon
```



Le programme affiche son message toutes les 5 secondes ; arrêt avec Ctrl+C (affiché ^C).

Le programme tourne comme demandé dans le cahier des charges : boucle infinie, message toutes les 5 secondes, sortie vidée immédiatement. Il est prêt pour le Job 2 (service systemd).